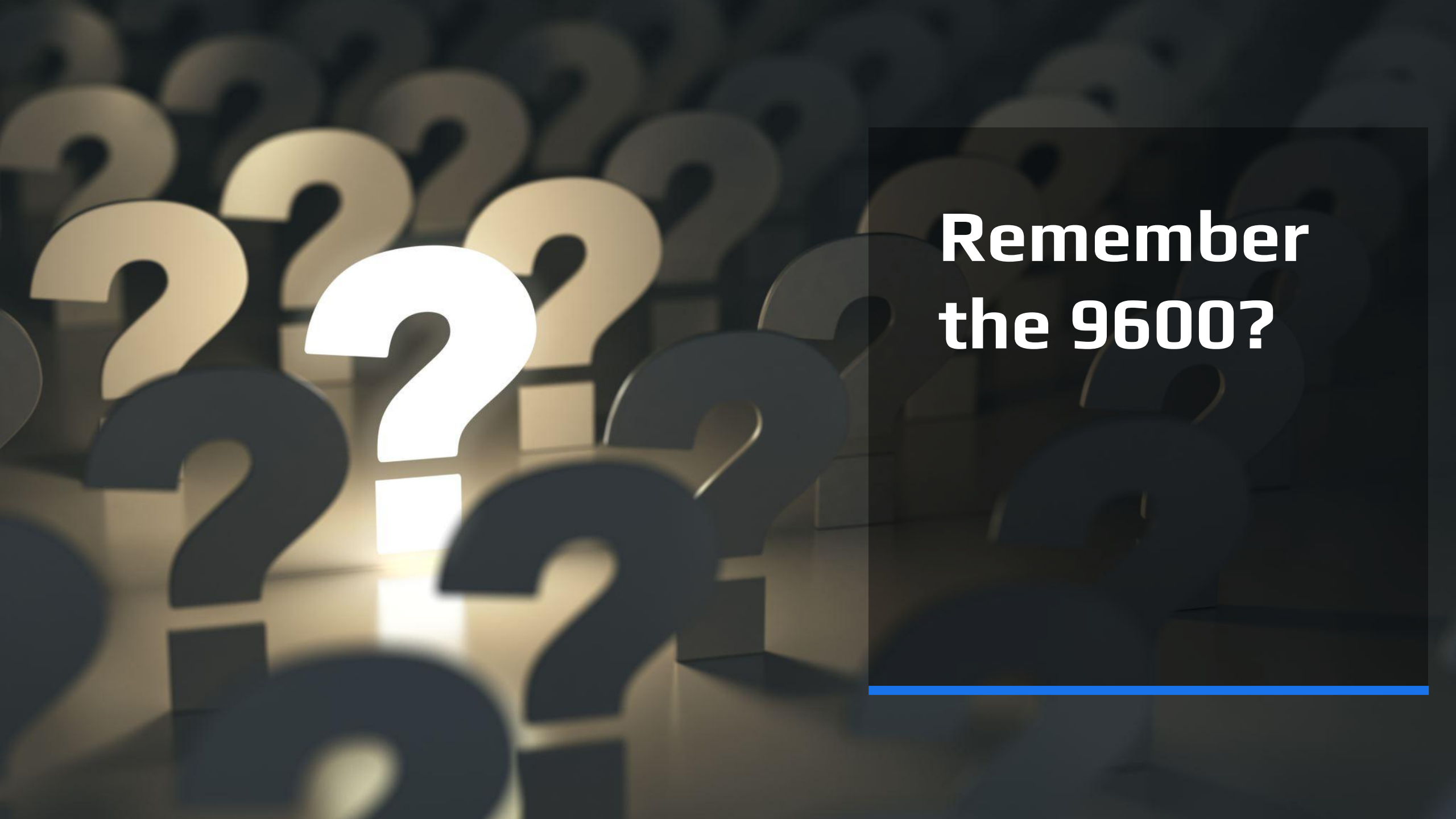


SERIAL COMMUNICATION

BY : BILAL AHMED


Remember
the 9600?

Background

Your robot has an Arduino, an ultrasonic sensor, a gyroscope module, and an SD card module — all needing to send data.

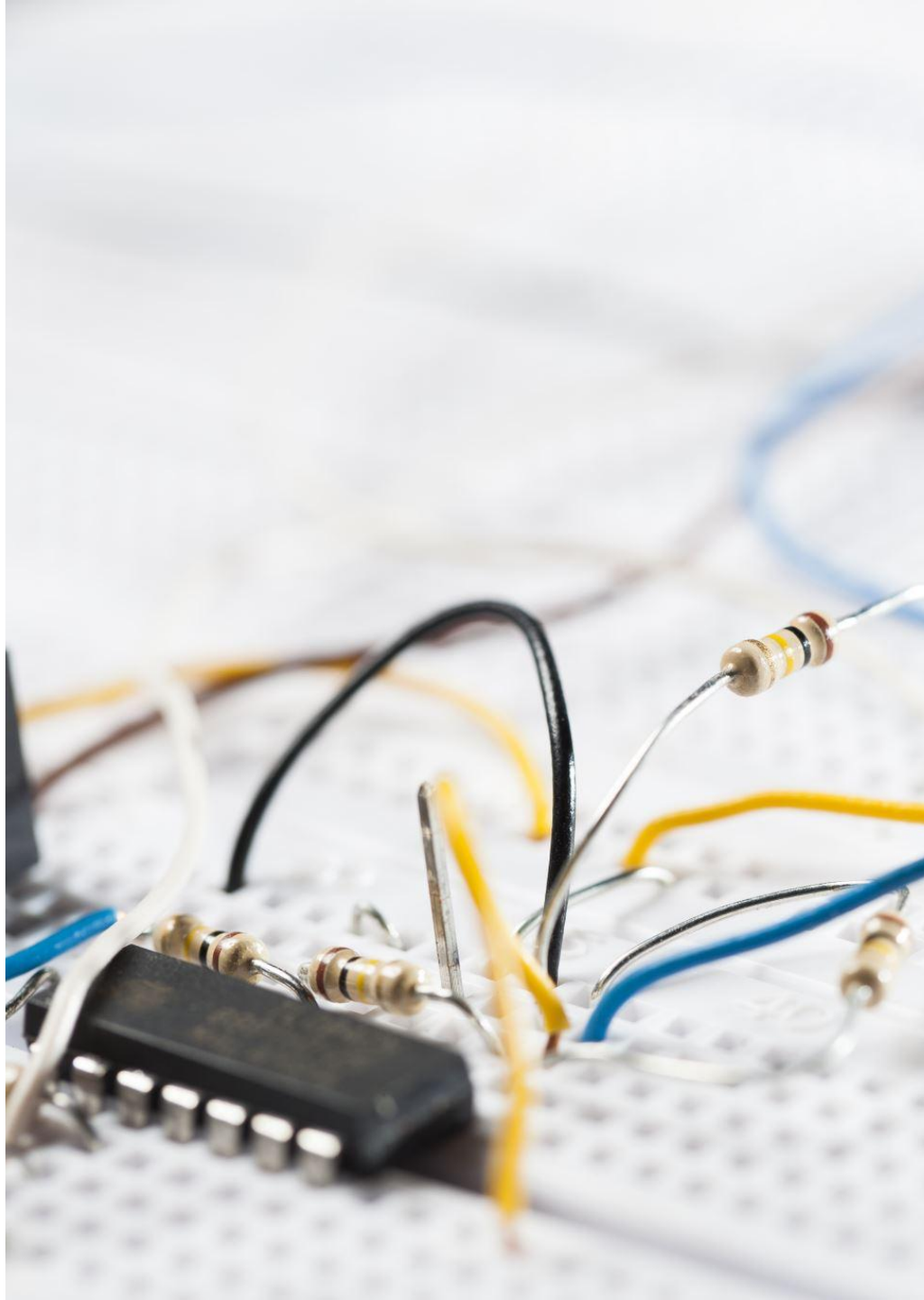
If you gave every single one its own dedicated set of wires and its own protocol, how many wires would your robot need, and how much code would you have to write?

Now — what if instead, engineers designed a small handful of *standard* communication protocols that most components agree to speak,

So you only need to learn 2–3 patterns to talk to almost anything.

Background

- There are a few standard languages these components speak — UART, SPI, and I2C are the big three you'll use constantly in robotics.
- Today we're starting with UART, since it's the simplest and the one you've already been using every time you plugged Arduino into your laptop



Three Protocols You need to know

Protocol	Wires Needed	Speed	Typical Use
UART	2 (TX, RX)	Moderate	GPS, Bluetooth, PC communication
SPI	4 (MOSI, MISO, SCK, SS)	Fast	SD cards, displays, high-speed sensors
I2C	2 (SDA, SCL) — shared by multiple devices	Moderate	Multiple sensors on one bus (IMU, RTC, etc.)

Your USB cable has only 4 wires. Your old parallel printer cable had 25. Which one transfers data faster today, and why do you think that is?

SERIAL PORT CABLES



USB PORT



RS 232 PORT (MALE)

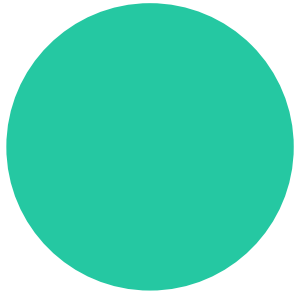
PARALLEL PORT CABLES



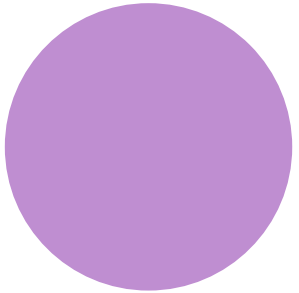
DB25

CENTRONICS 36

The Answer : USB serial



Because serial can run at much higher clock speeds



And there would be no issue of wires interfering with each other.

SERIAL COMMUNICATION

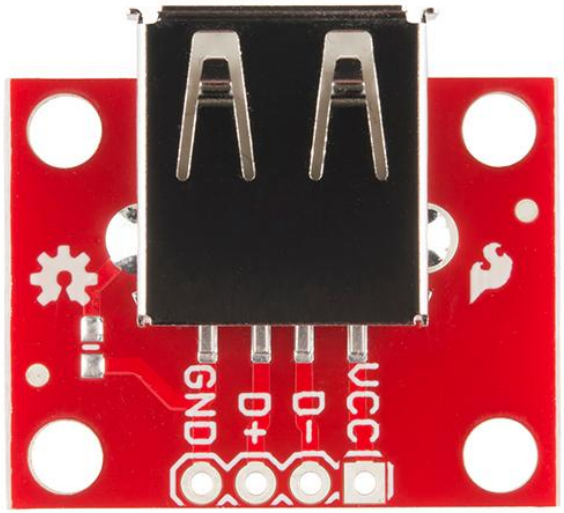
Serial communication sends data **one bit at a time**, over a single wire — like passing letters through a narrow mail slot instead of a wide door.

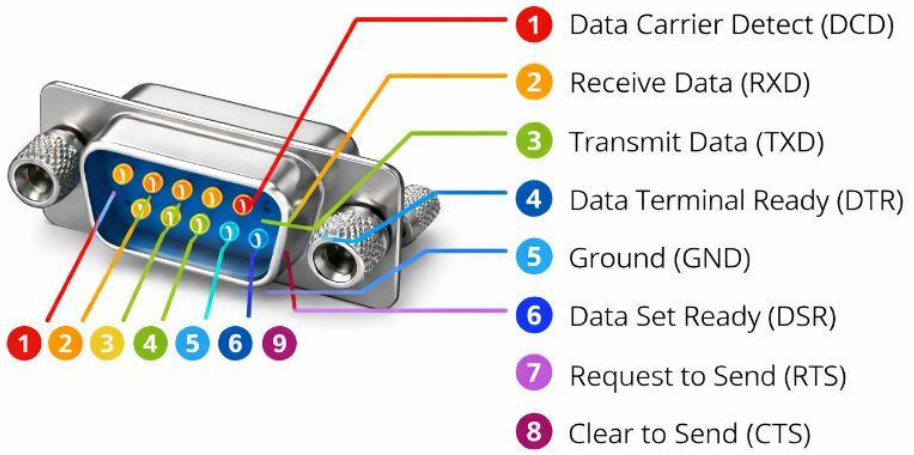
Contrast with **parallel communication**: multiple bits sent simultaneously over multiple wires (faster, but needs more wires and is prone to timing mismatch over long distances).

Real-world connection: Every time your phone connects to a Bluetooth speaker, or your laptop talks to a USB mouse, it's serial communication underneath

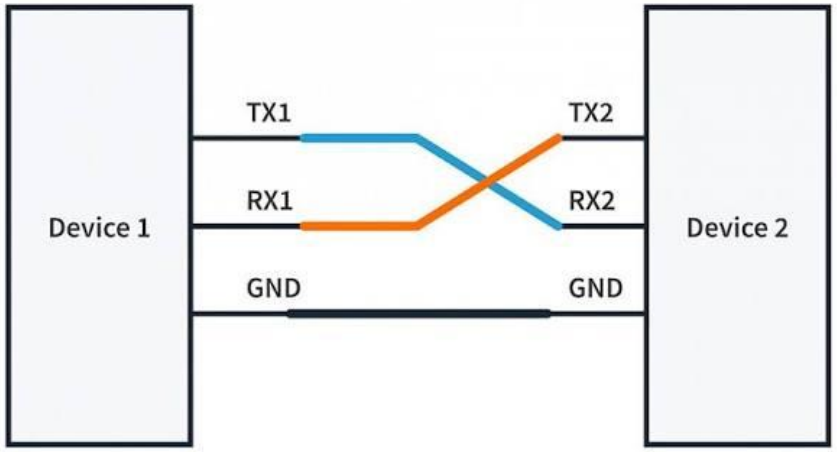
Parallel communication is mostly gone from modern devices because serial scales better with distance and cable simplicity.

Pinout of Serial Ports





UART — The Protocol Behind Arduino's Serial



UART = Universal Asynchronous Receiver/Transmitter.

Asynchronous means there's no shared clock signal between devices —

Both sides must simply agree in advance on the **speed** (baud rate) they'll communicate at.

Data travels on two wires: **TX** (transmit) and **RX** (receive) — always cross-connected (TX→RX, RX→TX).

Baud rate = bits transmitted per second (e.g., 9600 baud ≈ 9600 bits/sec).

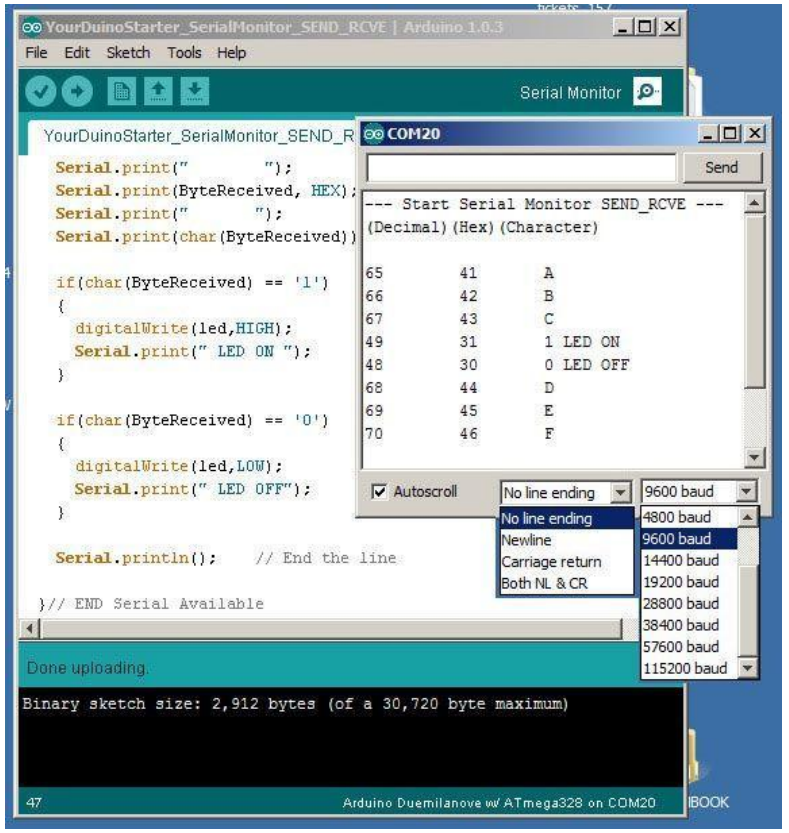
Both devices MUST use the same baud rate, or received data turns into garbage characters.

The Serial Monitor — Your Window Into the Program

Serial Monitor (Tools → Serial Monitor) displays whatever Arduino sends via Serial.print() / Serial.println().

It's essentially a **live window into your program's brain** — the single most useful debugging tool in embedded development.

Key settings that must match: **baud rate** (dropdown, bottom-right) and **line ending** (None / Newline / Carriage Return / Both).



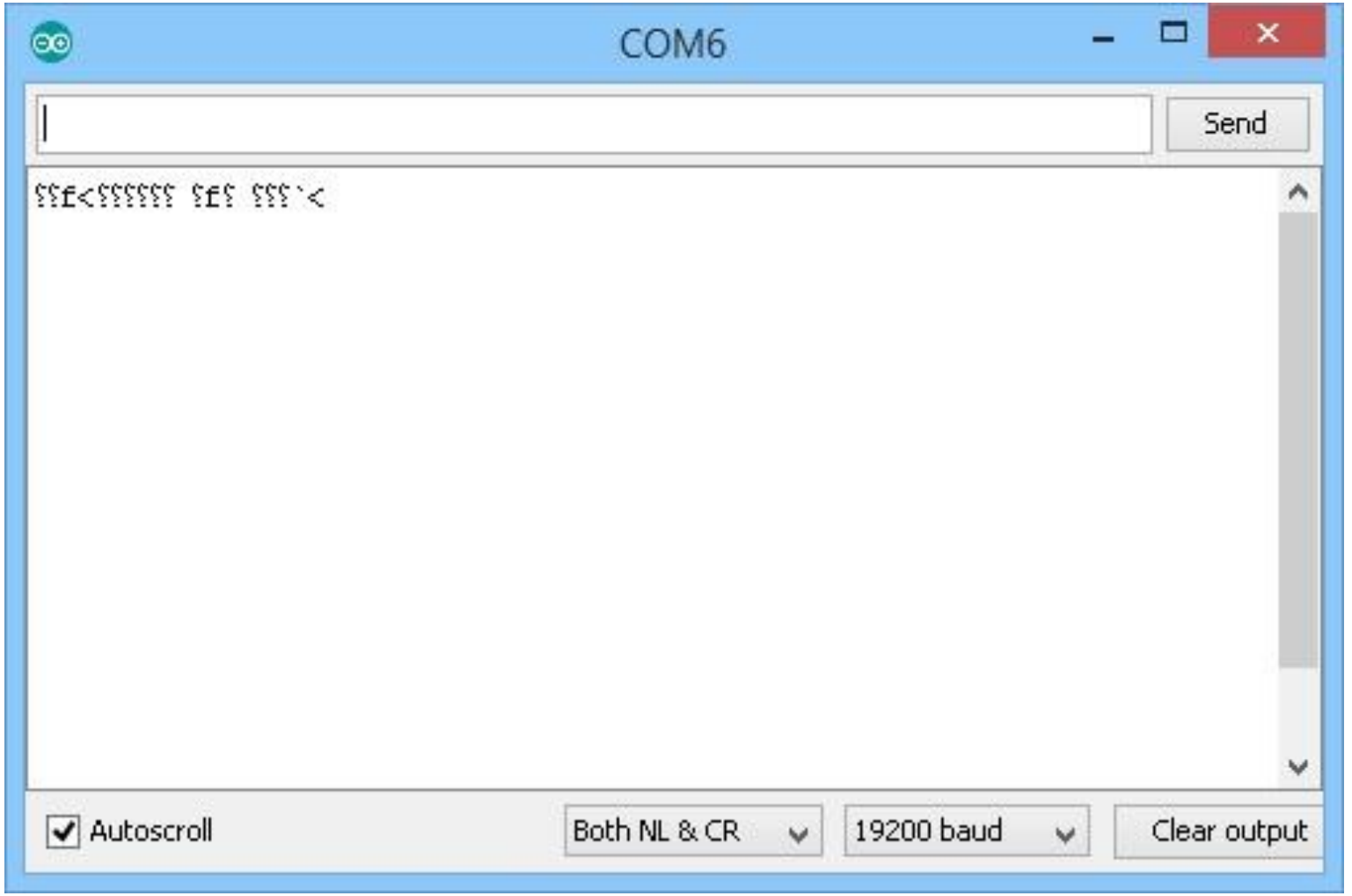
Sample Code

```
void setup() {
  Serial.begin(9600);
}


void loop() {
  Serial.println("Loop is running...");
  delay(500);
}
```

If Arduino sends at 9600 baud but your Python script or PC listens at 115200 baud, what will you see on screen?

This will happen...





Debugging With Serial — The "Print Statement" Superpower

- Before an oscilloscope, before a debugger — the simplest embedded debugging tool is: **print what the program is doing, at every important step.**
- Common debugging pattern: print variable values right before a suspicious line of code runs.
- **Real-world parallel:** professional embedded engineers call this "printf debugging“.
- Even NASA engineers have historically used log/print statements to debug flight software when live debugging hardware wasn't available.

Common Serial Debugging Pitfalls (Quick Reference)

Symptom	Likely Cause
Garbled/random characters	Baud rate mismatch between code and Serial Monitor
Nothing appears at all	Wrong COM port selected, or Serial.begin() missing
"Port already in use" error	Two programs (IDE + Python, etc.) trying to open the same port
Numbers look wrong/jumbled at high speed	Using parseInt() on a continuous fast stream instead of Serial.read()

Real-World Devices That Use UART/Serial

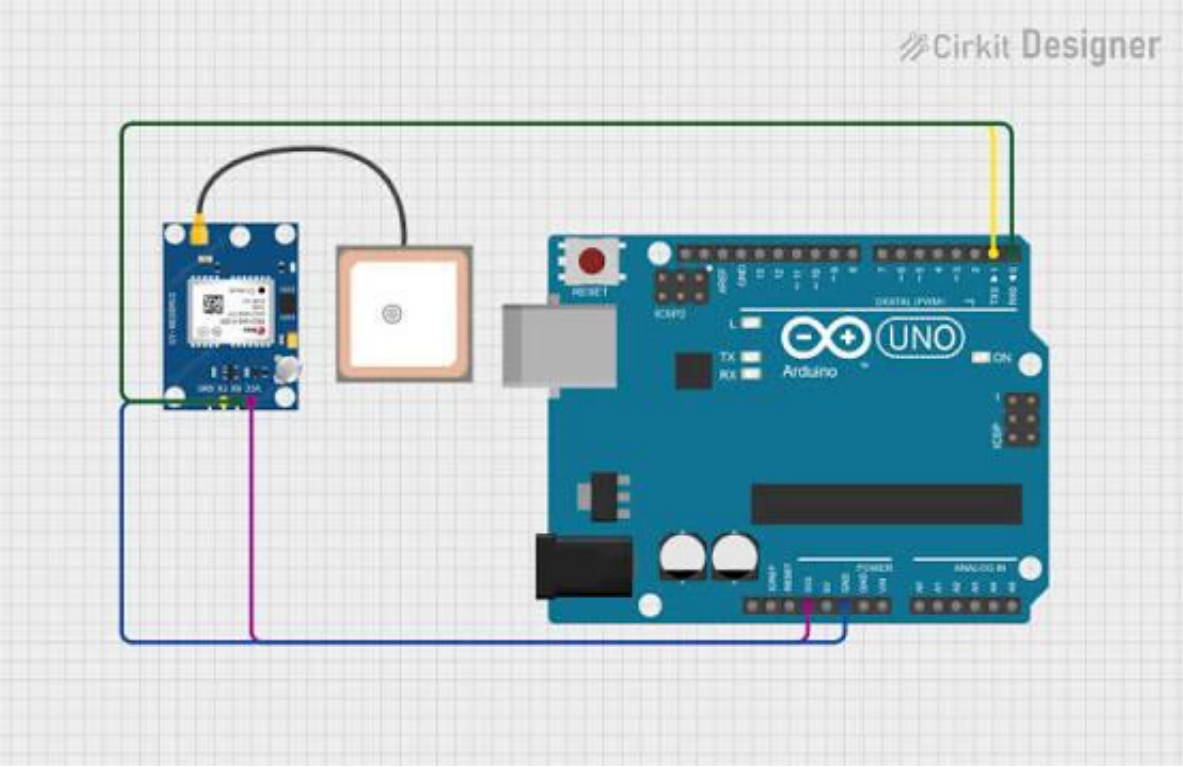
GPS modules — send location data as serial text strings (NMEA sentences).

Bluetooth modules (HC-05/HC-06) — communicate with Arduino purely over UART.

Industrial sensors & PLCs — RS-232/RS-485 serial links still run factories today.

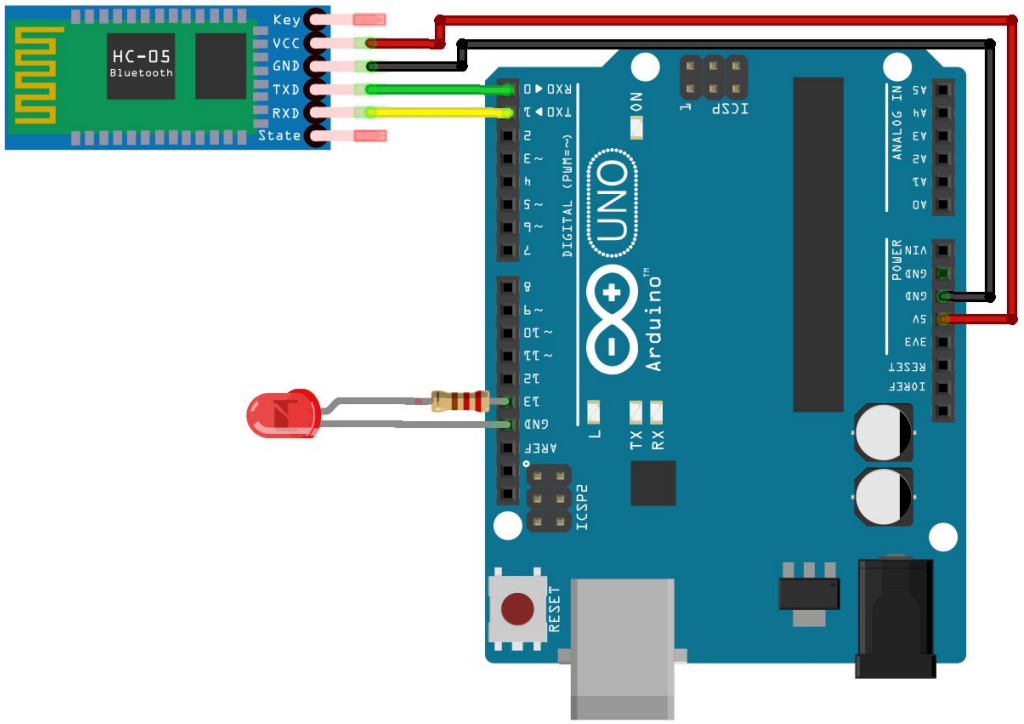
Debug ports on routers, embedded Linux boards — engineers plug into a serial console to fix a "bricked" device when nothing else works.

GPS — Autonomous Vehicles & Drones



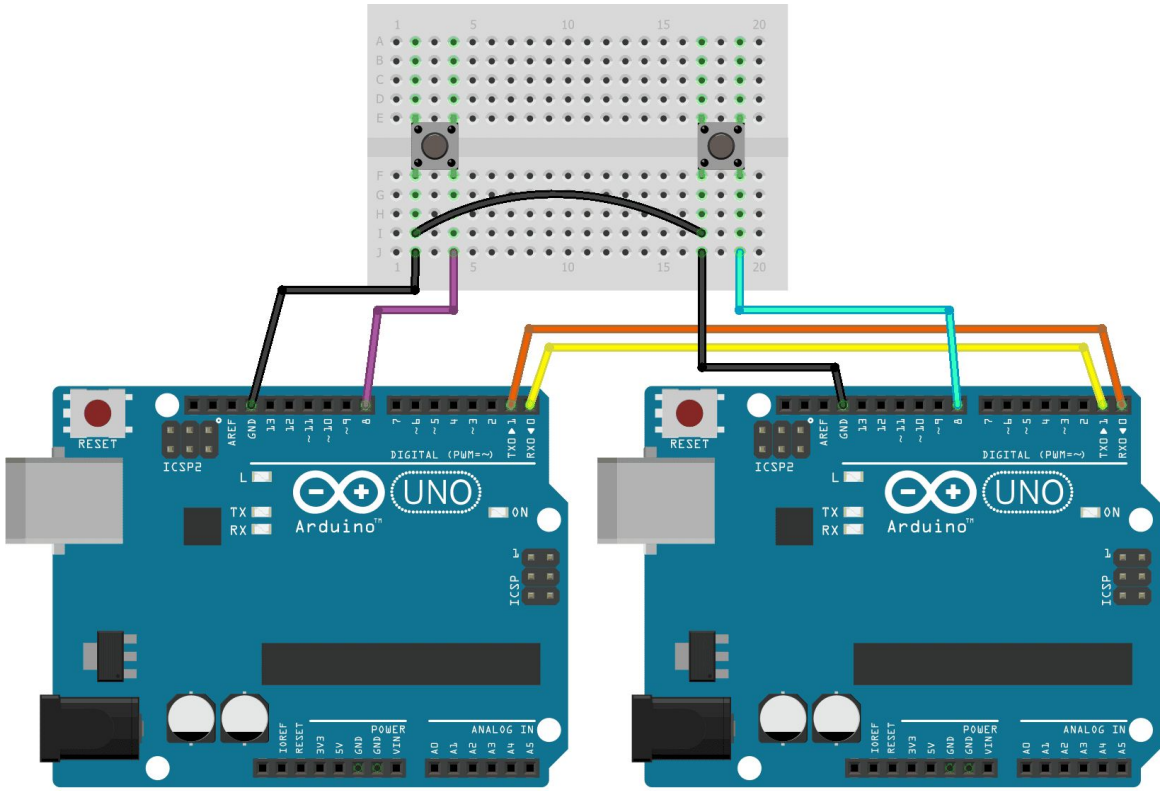
- GPS modules (NEO-6M, NEO-M8N) stream location data over UART as NMEA text sentences.
- **Autonomous vehicles/rovers:** GPS feeds coordinates over UART to the onboard computer for real-time localization.
- **Drones:** Flight controllers (Pixhawk/ArduPilot) use UART GPS data for waypoint navigation and return-to-home.

Bluetooth Control Modules



- HC-05/HC-06 Bluetooth modules connect to Arduino via UART.
- **Example:** Phone-controlled robotic arms/rovers — app commands travel over Bluetooth, then UART to Arduino, triggering motor movement.

Arduino to Arduino Communication



- Complex robots often split tasks across **two or more Arduinos** — one for sensing, one for actuation — to avoid overloading a single board's processing/timing.
- UART is the simplest way for two Arduinos to exchange data: connect **TX↔RX, RX↔TX, GND↔GND**.

Example — Sensor Board + Motor Controller Board

Code — Sensor Arduino (Sender):

```
void setup() {
  Serial.begin(9600);
  pinMode(A0, INPUT);
}

void loop() {
  int distance = analogRead(A0);
  Serial.println(distance);   // Send sensor value to
motor Arduino
  delay(100);
}
```

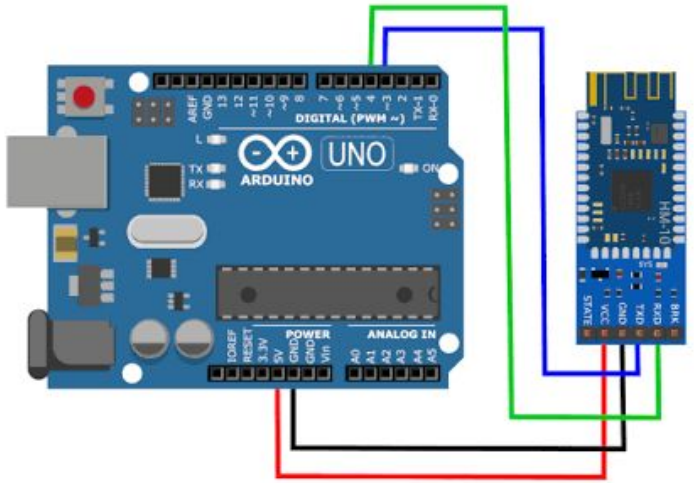
Code — Motor Arduino (Receiver):

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  if (Serial.available() > 0) {
    int distance = Serial.parseInt();
    if (distance < 200) {
      // stop or turn motors
    }
  }
}
```

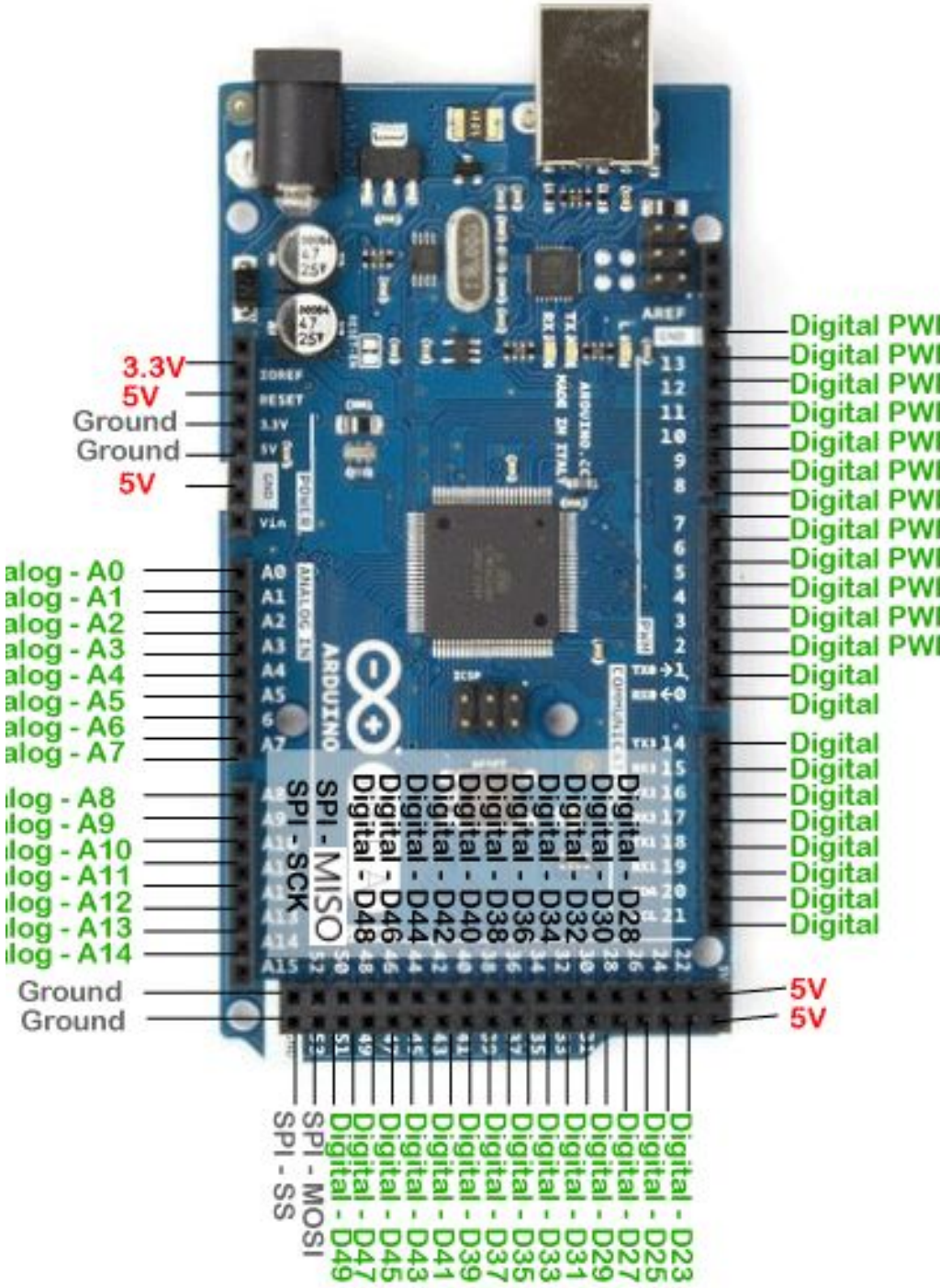
Hardware UART vs. Software Serial

- **The Core Limitation**
  - Arduino Uno has only **one hardware UART**, fixed to **pins 0 (RX) and 1 (TX)**.
  - These same pins are **shared with the USB connection** — used for uploading code and running Serial Monitor.
  - Using pins 0/1 for another device (like GPS or Bluetooth) conflicts with programming and debugging.
- **The Solution: Software Serial**
  - The **SoftwareSerial library** emulates a UART connection on **any two regular digital pins**, using software instead of dedicated hardware.
  - This frees pins 0/1 to stay reserved for **PC communication only**, while a second device communicates independently on different pins.



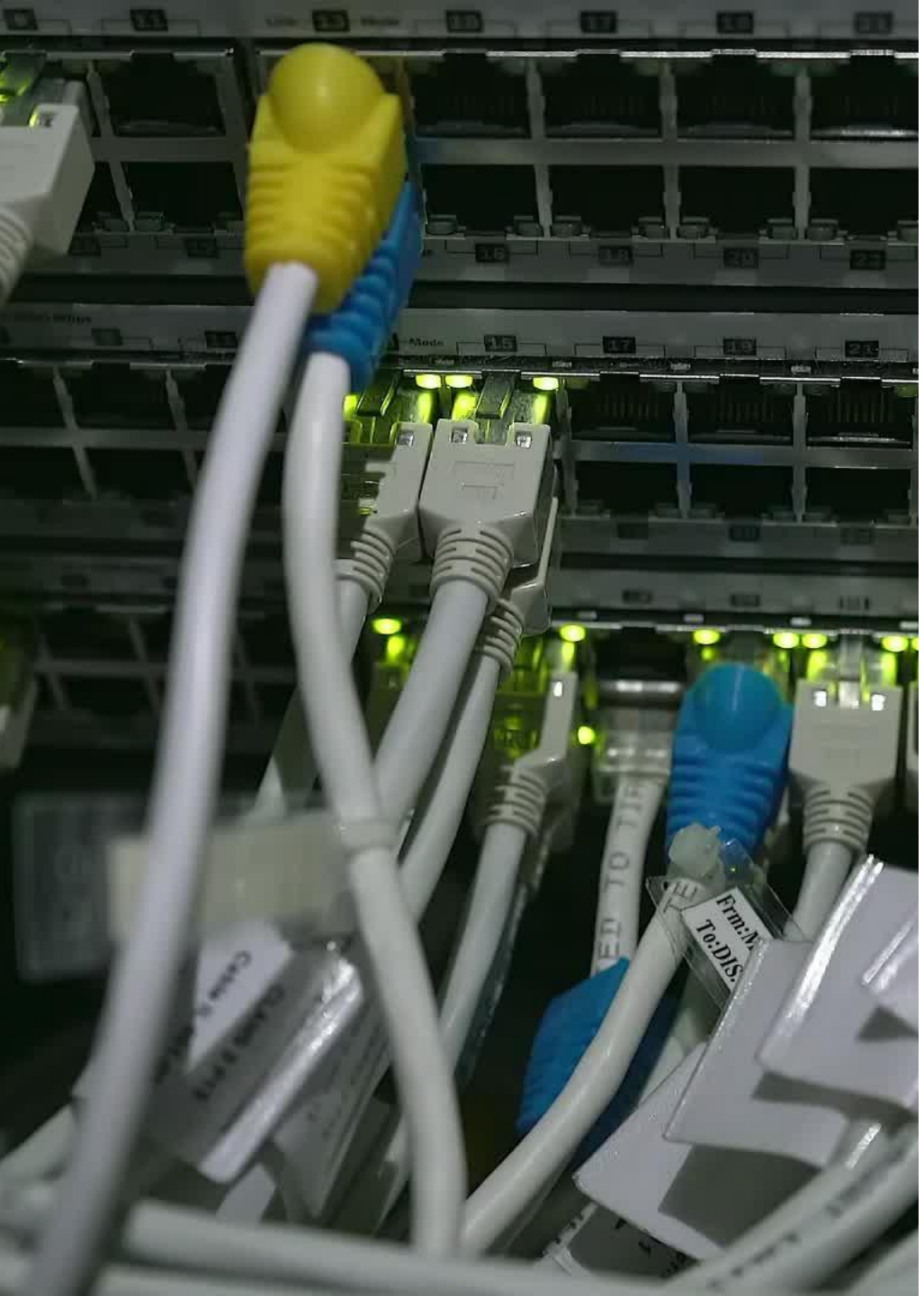
Boards With Multiple Hardware UARTs

- Not all boards have this limitation — some have **multiple built-in hardware UARTs**, avoiding the need for SoftwareSerial entirely.
- **Arduino Mega 2560:** 4 hardware UARTs — Serial, Serial1, Serial2, Serial3.
- **ESP32:** 3 hardware UARTs available.
- On these boards, **pin 0/1 stays free for USB**, while Serial1/Serial2 connect directly to other devices — no software emulation needed.



Comparison

Feature	Software Serial (Uno)	Hardware UART (Mega/ESP32)
Pins used	Any digital pins	Dedicated fixed pins per port
Speed/reliability	Lower, can miss data at high baud rates	Full speed, reliable
Number of extra devices	Usually 1 at a time	Multiple simultaneously
Why needed	Only 1 UART exists on the board	Board already has spare



Other Options of Serial Communications in Arduino

- UART is not only Serial option communication in Arduino
- I2C and SPI can also be used for serial communication of devices with Arduino.
- So what is the difference?

Difference : Their architecture

Protocol	Serial?	Synchronous?	Clock Wire?
UART	Yes	Asynchronous — no shared clock, both sides pre-agree on baud rate	No
SPI	Yes	Synchronous — dedicated clock line keeps both sides perfectly timed	Yes (SCK)
I2C	Yes	Synchronous — dedicated clock line, plus device addressing	Yes (SCL)

Quick Distinction Recap

UART — 2 wires (TX, RX), asynchronous (no clock — timing agreed in advance via baud rate), point-to-point only (one device to one device).

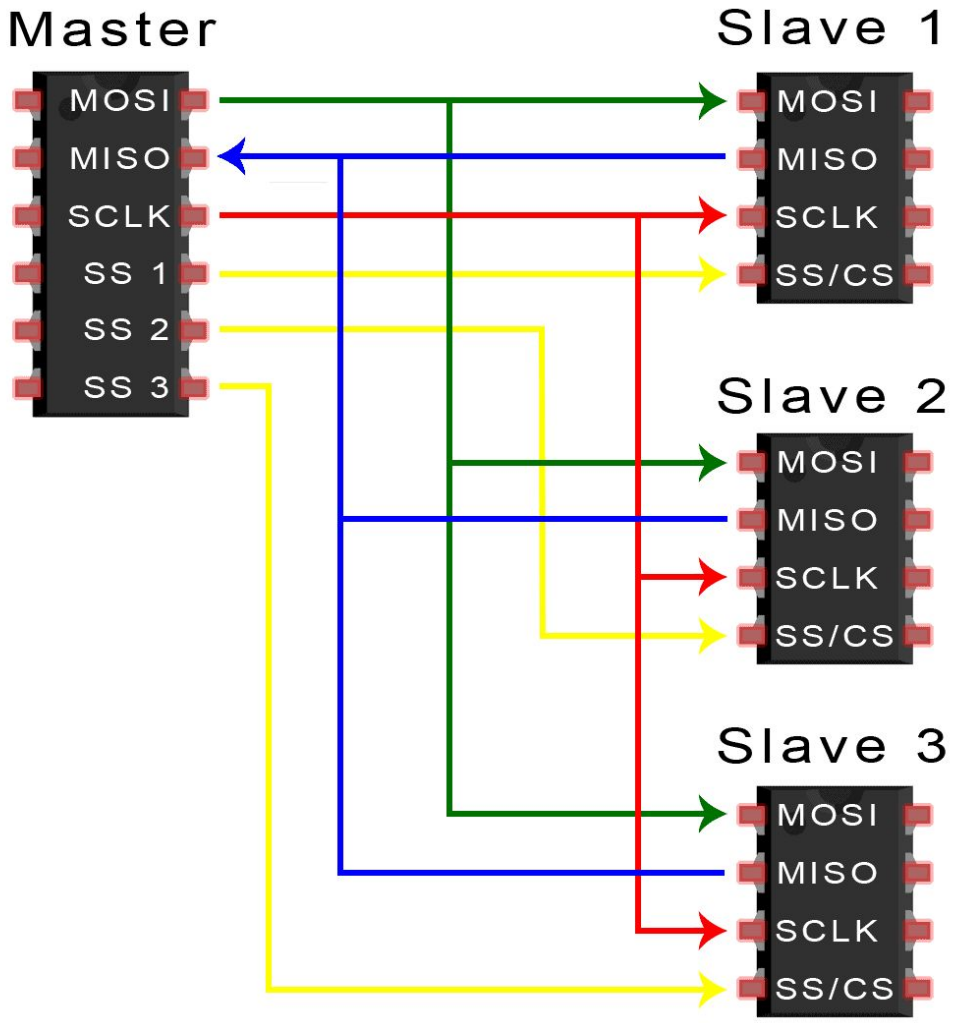
SPI — 4 wires (MOSI, MISO, SCK, SS), synchronous (clock wire keeps devices in lockstep), fast, supports multiple devices via separate Slave Select (SS) lines.

I2C — 2 wires (SDA, SCL), synchronous, supports **multiple devices on the same two wires simultaneously**, distinguished by unique addresses (unlike SPI, which needs a separate wire per device).

SPI Communication: Remember programmer topic where we needed to directly write program to Atmega328 microcontroller chip?

SPI

- **SPI (Serial Peripheral Interface)** — a **synchronous** serial protocol using a dedicated clock line to keep sender and receiver perfectly in sync.
- Much faster than UART or I2C — commonly runs at several MHz, sometimes tens of MHz.
- Follows a strict **Master–Slave** architecture: one master device controls the timing; one or more slaves respond.



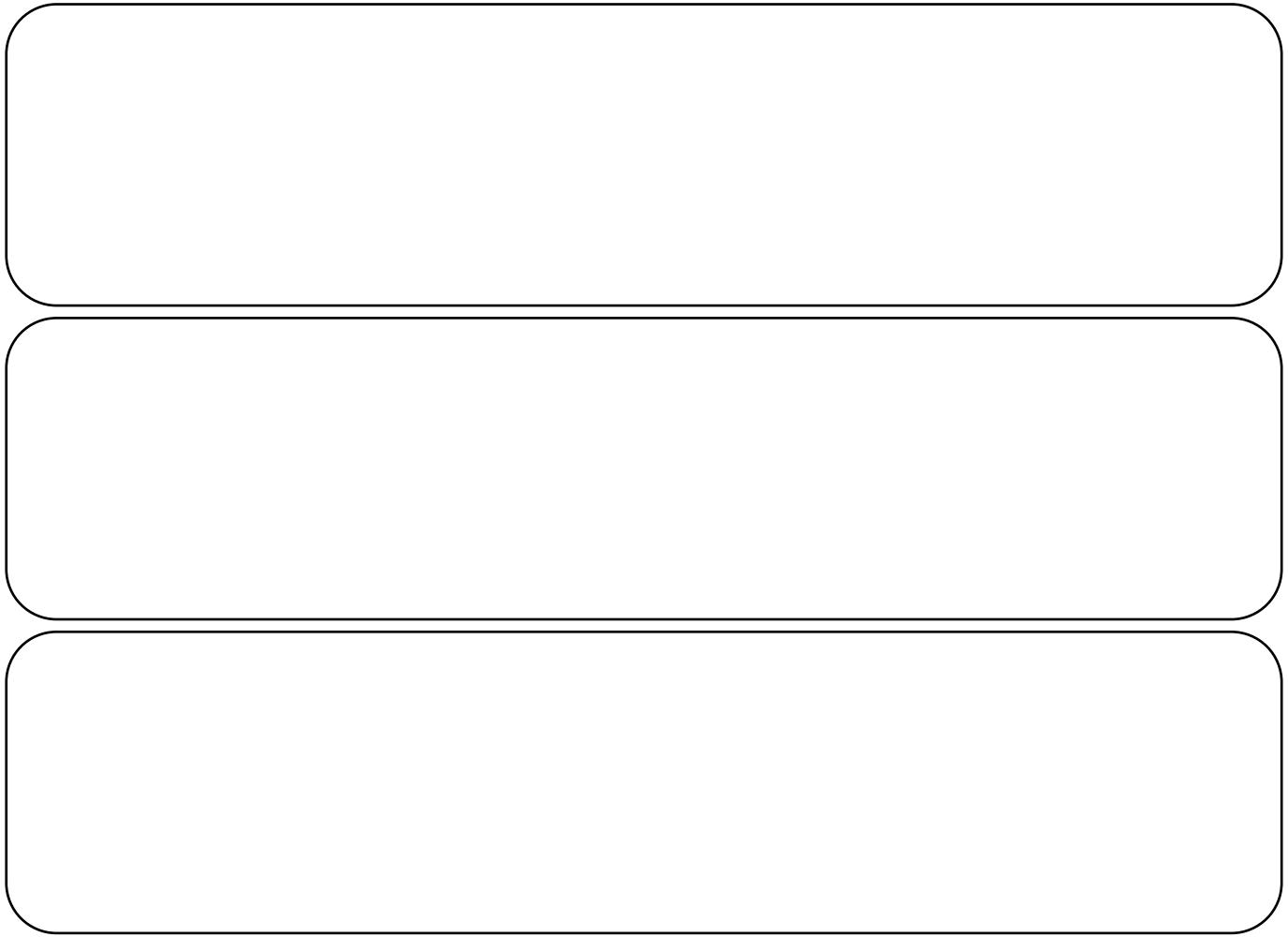
SPI Architecture — The 4 Wires

Wire	Full Name	Function
MOSI	Master Out, Slave In	Master sends data to slave
MISO	Master In, Slave Out	Slave sends data back to master
SCK	Serial Clock	Master generates timing pulses — keeps both sides synchronized
SS / CS	Slave Select / Chip Select	Master pulls this LOW to "wake up" a specific slave for communication

SPI: Full Duplex

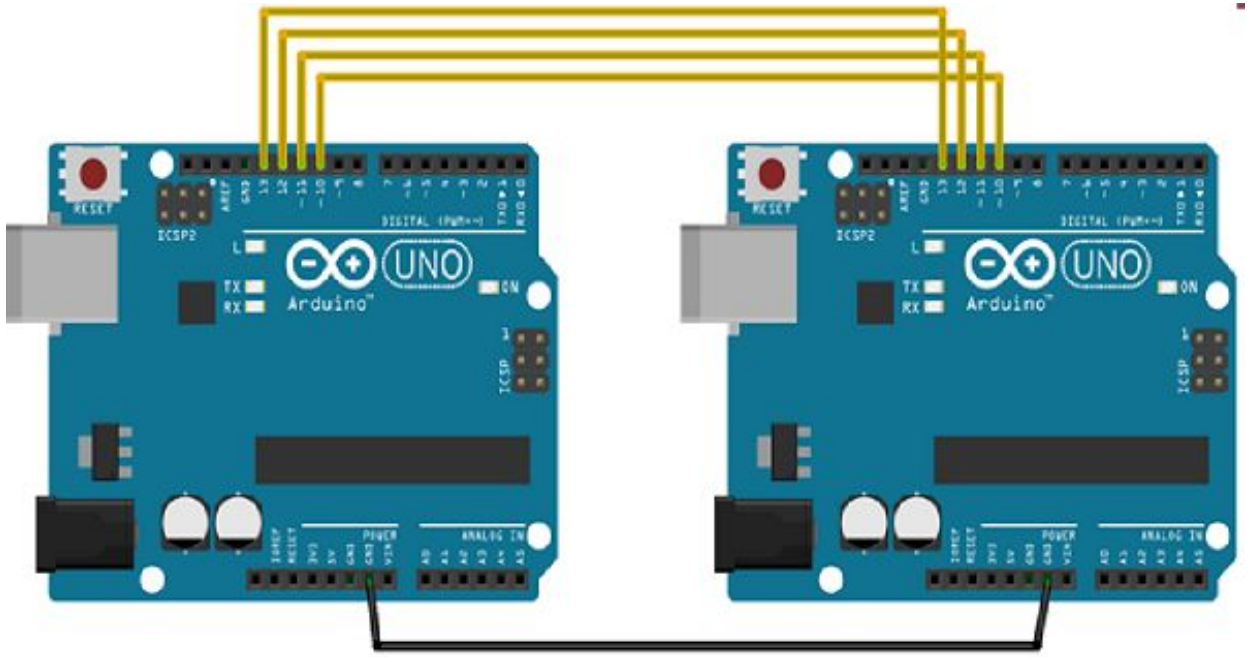


Multiple Devices on One SPI Bus



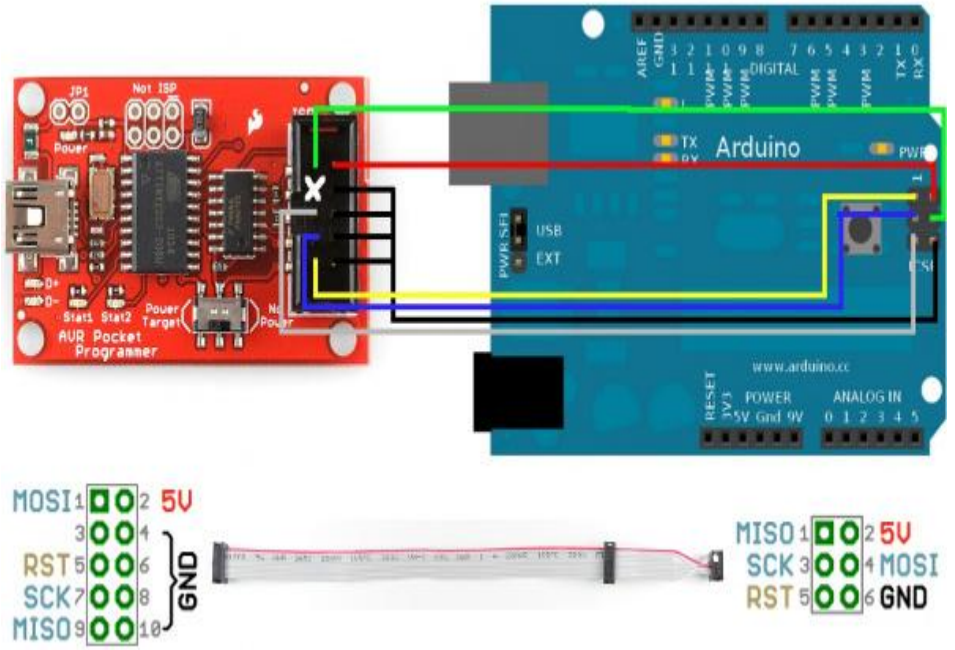
SPI on Arduino Uno — Fixed Pins

Function	Arduino Uno Pin
MOSI	Pin 11
MISO	Pin 12
SCK	Pin 13
SS (default)	Pin 10



SPI's Role in Programming Chips — ICSP & AVR Studio

- SPI isn't just for sensors — it's also how **AVR microcontrollers get physically programmed**.
- The **ICSP (In-Circuit Serial Programming) header** on Arduino boards uses the **exact same SPI protocol** (MOSI, MISO, SCK) to write a bootloader or program directly onto a blank ATmega328P chip.
- **AVR Studio / Atmel Studio** (Microchip's official IDE) uses a **hardware ISP programmer** (e.g., USBasp, AVRISP mkII) connected via this same SPI interface to burn firmware
- This is exactly how the breadboard Arduino build (from your earlier lab) gets its bootloader before it can even run sketches.

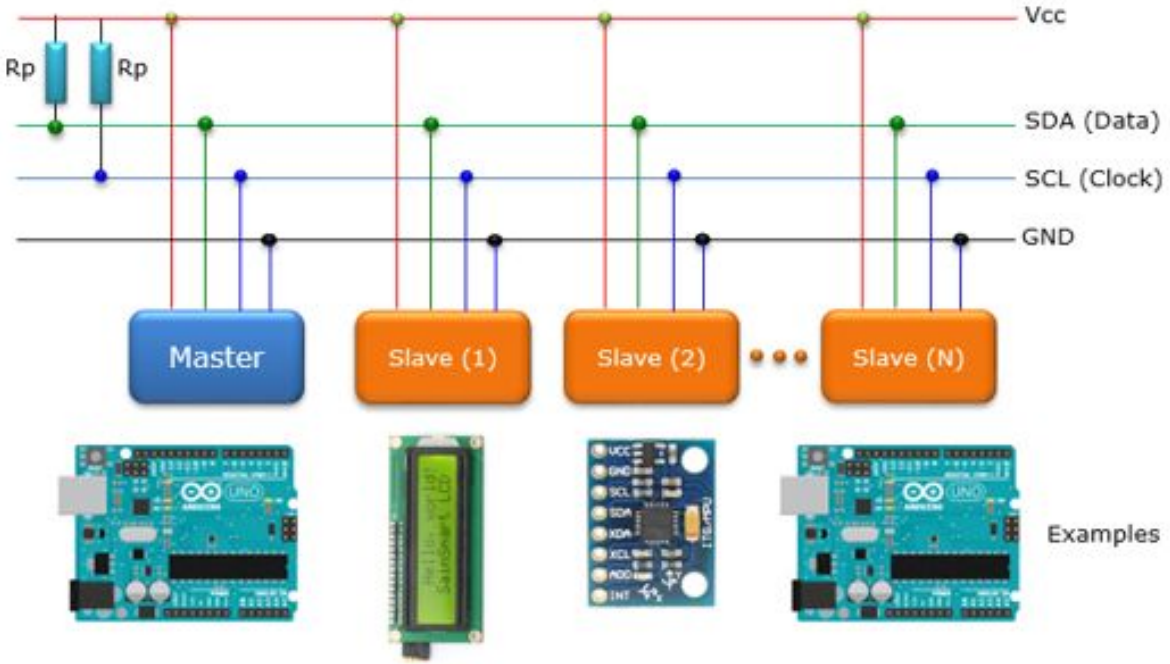


SPI in Robotics Applications

- **SD card modules** — logging sensor data, storing maps/paths for autonomous robots.
- **High-speed IMUs & gyroscopes** — SPI's speed is preferred over I2C when robots need very fast, high-frequency motion data (e.g., drone flight controllers reading gyro data thousands of times per second).
- **TFT/OLED displays** — robot status screens, live sensor readouts.
- **RF modules (nRF24L01)** — common in wireless robot control and simple swarm/multi-robot communication experiments.
- **Multiple chained sensors** — e.g., several SPI-based distance or pressure sensors read rapidly in sequence for real-time obstacle mapping.

I2C — Inter-Integrated Circuit

I2C



- **I2C (Inter-Integrated Circuit)**, pronounced "I-squared-C" — a **synchronous** serial protocol using just **2 wires** to connect **multiple devices** on the same bus.
- Developed by Philips for connecting multiple chips on a single circuit board with minimal wiring.
- Unlike SPI (one SS wire per device), I2C devices share the same two wires and are told apart by a unique **address** instead.

I2C Architecture — The 2 Wires

Wire	Full Name	Function
SDA	Serial Data	Carries actual data, both directions
SCL	Serial Clock	Master generates timing pulses — keeps all devices synchronized

- **Master–Slave (with multi-master support):** typically one master initiates all communication, but I2C also supports multiple masters sharing the same bus (more advanced use case).

Device Addressing — How Multiple Devices Share 2 Wires

- Every I2C slave device has a **unique 7-bit address** (some support 10-bit) — either fixed by the manufacturer or configurable via address pins on the chip.
- The master sends the target device's address first; only the matching device responds — every other device on the bus stays silent.
- **Example:** An IMU might use address 0x68, an RTC module 0x68 or 0x51, an OLED display 0x3C — all can share the exact same SDA/SCL wires simultaneously, as long as addresses don't collide.

I2C on Arduino Uno — Fixed Pins

Function	Arduino Uno Pin
SDA	A4
SCL	A5

- Like SPI, these pins are **fixed in hardware** and shared by all I2C devices connected to the board — no reassignment possible on Uno.

I2C in Robotics Applications

- **IMUs (MPU6050, etc.)** — orientation/motion sensing for balancing robots, drones.
- **RTC (Real-Time Clock) modules** — timestamping data-logging robots, scheduled tasks.
- **OLED/LCD displays** — showing robot status, sensor readings, battery level.
- **Multiple identical sensors on one bus** — e.g., several distance sensors around a robot's perimeter, each with a distinct address, all sharing just 2 wires instead of dedicating separate pins to each.
- **Sensor hub boards** — many robotics "breakout" sensor modules (pressure, color, proximity) default to I2C specifically because it scales to many devices with minimal wiring complexity.

Summary

Property	UART	SPI	I2C
Wires	2 (TX, RX)	4 (MOSI, MISO, SCK, SS)	2 (SDA, SCL)
Clock	None (asynchronous)	Yes (SCK)	Yes (SCL)
Duplex	Full-duplex	Full-duplex	Half-duplex
Devices per bus	1 (point-to-point)	Multiple (1 SS wire each)	Multiple (address-based)
Speed	Moderate	Fastest	Moderate
Addressing needed?	No	No (uses SS lines)	Yes (unique device address)
Extra wiring per device	—	+1 wire (SS) per device	None — shared bus
Arduino Uno pins	0 (RX), 1 (TX)	11, 12, 13 (+ SS)	A4 (SDA), A5 (SCL)